In this chapter, you'll see:

- The importance of carefully constructing an environment for each spec
- How to provide test doubles to the code you're testing
- What the most common pitfalls are and how to avoid them
- How to improve your code by applying design feedback from your test doubles

CHAPTER 15

Using Test Doubles Effectively

Over the previous two chapters, you've tried out mocks, stubs, spies, and null objects. You've learned which situations each is best for. You've also seen how to configure their behavior, and how to check that a test double is called correctly.

Now we'd like to talk about the trade-offs. Although we frequently use doubles in our specs, we'll be the first to acknowledge that doing so incurs some risk. Here are some of the problems you can run into:

- Code that passes the tests but fails in production, because the test doubles don't behave enough like the real thing
- Brittle tests that fail after a refactoring, even though the new code is working correctly
- Do-nothing tests that only end up checking your doubles

In this chapter, we're going to show you how to use test doubles *effectively*, meaning that the benefits to your project outweigh these risks.

Constructing Your Test Environment

People new to test doubles often ask how much behavior to fake. After all, if a test is too far removed from reality, it won't give you a clear idea of how the code will run in the real world. We've definitely seen tests that have gone overboard with mocks, stubs, spies, and null objects. It can be difficult to know where to draw the line.

To cut through the confusion, we like to think of a test suite as a laboratory for carefully exercising code. Your test doubles are your scientific instruments. They help you create the environment needed for each experiment. You use them to control the factors you care about, and nothing else.